

Skip List

–Memory Allocation

According to 64 – bit Architecture

Basically, we are dealing with integer variable and pointer variable

In 64 – bit architecture :

- *A integer variable is of 4 bytes.*
- *A pointer variable is of 8 – bytes.*

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};  
struct Node *n = createNode(key, rlevel);
```

Formation of Struct Node – An Memory Layout Example

```
n->key = key;  
  
    // Allocate memory for the forward pointer array (level + 1  
elements)  
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

Say,

int key starts with address : $(0x5000)_{16}$

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = (20480)_{10} \Rightarrow \frac{(20480)_{10}}{4 \text{ bytes}} \text{ i.e.}$$

$$\frac{(20480)_{10}}{(4)_{10}} = 5120_{10} \text{ and remainder : } 0, \text{ hence divisible by 4 bytes,}$$

$\therefore$ Data value will get store in $(0x5000)_{16}$.

But as ,int key is of 4 bytes it will hold (0 – 3)offset.

$$(0x5000)_{16} - \text{'0' offset i.e. } (0x5000 + 0) = (0x5000)_{16} \\ = \text{first byte of the integer(key)}$$

$$(0x5001)_{16} - \text{'1' offset i.e. } (0x5000 + 1) = (0x5001)_{16} \\ = 2^{\text{nd}} \text{ byte of the integer(key)}$$

$$(0x5002)_{16} - \text{'2' offset i.e. } (0x5000 + 2) = (0x5002)_{16} \\ = 3^{\text{rd}} \text{ byte of the integer(key)}$$

$$(0x5003)_{16} - \text{'3' offset i.e. } (0x5000 + 3) = (0x5003)_{16} \\ = 4^{\text{th}} \text{ byte of the integer(key)}$$

Say, *key* is = 20

Decimal = $(20)_{10}$

Hexadecimal = $(20)_{10} = (0x14)_{16}$

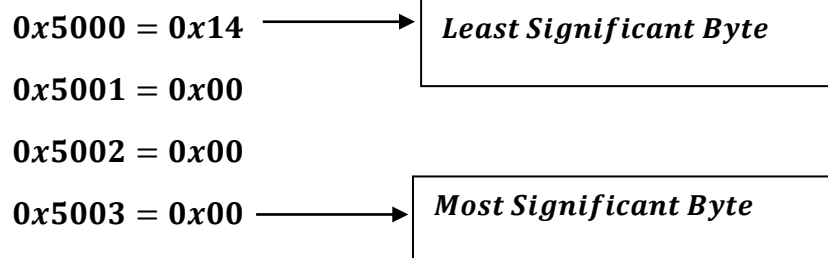
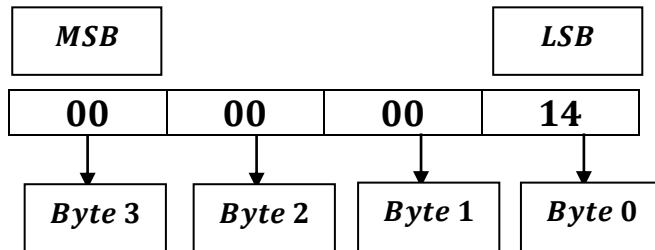
Hex: 0x00000014

Note : 0x00000014 has 8 hexadecimal digits.

Since 1 hexadecimal digit = 4 bits.

$$\Rightarrow 8 \times 4 \text{ bits} = 32 \text{ bits.}$$

$\Rightarrow 32 \text{ bits} \div 8 = 4 \text{ bytes} .$



As we know everything is stored in Memory address at binary format.

$0x14 = 00010100_2$ [8 bits]

$[00000000 \ 00000000 \ 00000000 \ 00010100]_2 = 20_{10}$ [8 × 4 = 32 bits]

$0x5000 = 00010100$ [8 bits]

$0x5001 = 00000000$ [8 bits]

$0x5002 = 00000000$ [8 bits]

$0x5003 = 00000000$ [8 bits]

Next, we have, to start with Address : 0x5004 and we have pointer variable Node ** forward , which will store 8 byte value.

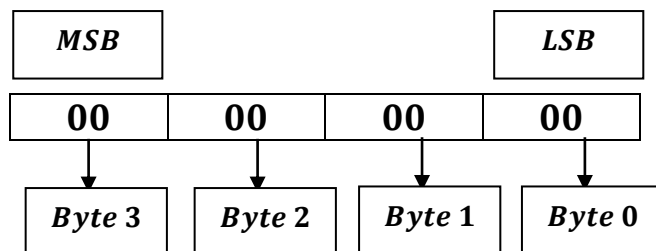
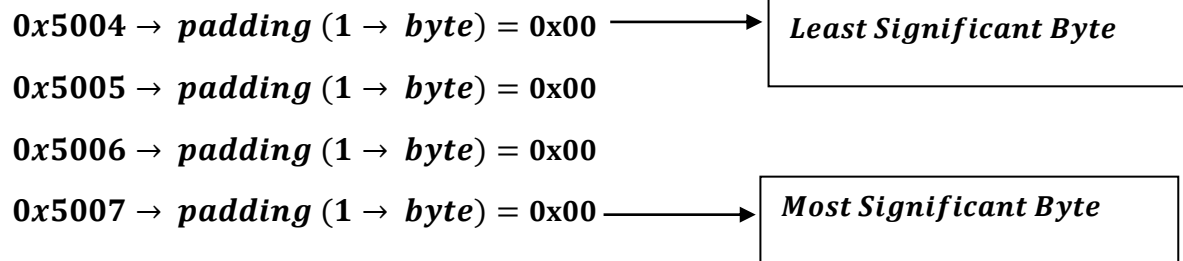
$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 4 \times 16^0 = (20484)_{10}$$

$$\Rightarrow \frac{(20484)_{10}}{8 \text{ bytes}} \text{ i.e. } \frac{(20484)_{10}}{(8)_{10}} = 2560_{10} \text{ and}$$

remainder : 4 , hence not divisible by 8 bytes .

This will lead to extra 4 bytes padding.

*[Earlier we have started with address divisible by 4 bytes ,
hence next address divisible by 8 bytes after $(8 - 4 = 4)$ i.e. 4 bytes.]*



Represented in 8 bits as:

$$0x5004 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5005 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5006 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

$$0x5007 \rightarrow \text{padding } (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$

*Next, we have, to start with Address : 0x5008 and we have pointer variable Node ** forward , which will store 8 byte value.*

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 8 \times 16^0 = (20488)_{10}$$

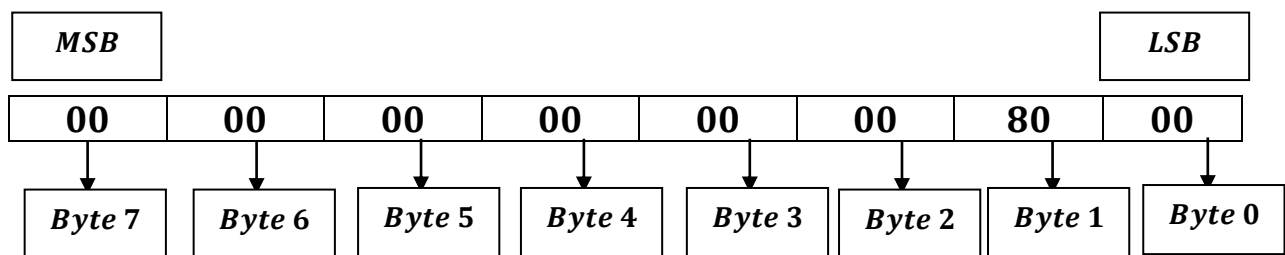
$$\Rightarrow \frac{(20488)_{10}}{8 \text{ bytes}} \text{ i.e. } \frac{(20488)_{10}}{(8)_{10}} = 2561_2 \text{ and}$$

remainder : 0 , hence divisible by 8 bytes .

And it will store : 0x8000 [8 byte address]

$$\text{Note: } (0x8000)_{16} = (1000 \ 0000 \ 0000 \ 0000)_2$$

$$0x8000 = 0x00 \ 00 \ 00 \ 00 \ 00 \ 00 \ 80 \ 00 \text{ [8 hexadecimal digits]}$$



0x5008 → forward double pointer (1 → byte) = 0x00 →

0x5009 → forward double pointer (1 → byte) = 0x80

0x500A → forward double pointer (1 → byte) = 0x00

0x500B → forward double pointer (1 → byte) = 0x00

0x500C → forward double pointer (1 → byte) = 0x00

0x500D → forward double pointer (1 → byte) = 0x00

0x500E → forward double pointer (1 → byte) = 0x00

0x500F → forward double pointer (1 → byte) = 0x00 →

*Least
Significant
Byte*

*Most
Significant
Byte*

$$\text{Note: } (0x80)_{16} = 1000 \ 0000_2$$

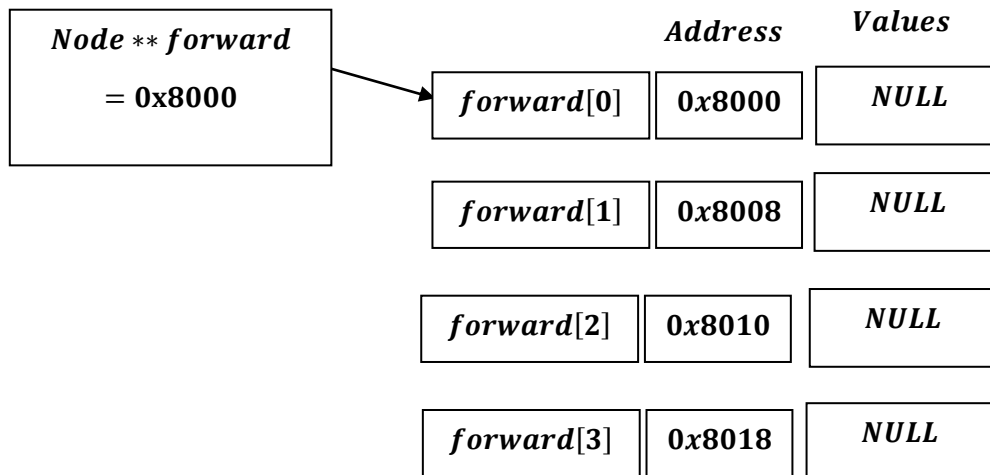
Represented in 8 bits as:

0x5008 → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x5009 → *forward double pointer* (1 → *byte*) = 1000 0000 [8 bits]
0x500A → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500B → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500C → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500D → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500E → *forward double pointer* (1 → *byte*) = 00000000[8 bits]
0x500F → *forward double pointer* (1 → *byte*) = 00000000[8 bits]

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x5000 – 0x5003	0 – 3	4 bytes	<i>Integer : key</i>	<i>Integer value(Key)</i>
0x5004 – 0x5007	4 – 7	4 bytes	<i>padding</i>	<i>Empty space (alignment)</i>
0x5008 – 0x500F	8 – 15	8 bytes	<i>double pointer</i> $\left(\begin{smallmatrix} \text{pointer} \\ \text{to} \\ \text{Pointer} \end{smallmatrix} \right)$ <i>variable named Forward</i>	<i>Address of forward[0] which is a single pointer variable.</i>

Now ,when we come to:



0x8000 is divisible by 8 (byte)hence:

0x8000 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8001 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8002 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8003 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8004 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8005 → forward single pointer[forward[0]](1 → byte) = 0x00

0x8006 → forward single pointer[forward[0]](1 → byte) = 0x00

0x8007 → forward single pointer[forward[0]] (1 → byte) = 0x00

Again, 0x8008 is divisible by 8 (bytes)hence:

$0x8008 \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8009 \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800A \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800B \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800C \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800D \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800E \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$
 $0x800F \rightarrow \text{forward single pointer}[\text{forward}[1]] (1 \rightarrow \text{byte}) = 0x00$

Again, 0x8010 is divisible by 8 (bytes) hence:

$0x8010 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8011 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8012 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8013 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8014 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8015 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8016 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8017 \rightarrow \text{forward single pointer}[\text{forward}[2]] (1 \rightarrow \text{byte}) = 0x00$

Again, 0x8018 is divisible by 8 (bytes) hence:

$0x8018 \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x8019 \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801A \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801B \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801C \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$
 $0x801D \rightarrow \text{forward single pointer}[\text{forward}[3]] (1 \rightarrow \text{byte}) = 0x00$

$$0x801E \rightarrow \textit{forward single pointer}[\textit{forward}[3]](1 \rightarrow \textit{byte}) = 0x00$$

0x801F $\rightarrow$ *forward single pointer*[*forward*[3]] (1 $\rightarrow$ *byte*) = **0x00**

$0x8000 \rightarrow \text{forward single pointer}[\text{forward}[0]] (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8001 \rightarrow \textit{forward single pointer}[\textit{forward}[0]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

$0x8002 \rightarrow \text{forward single pointer}[\text{forward}[0]] \ (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8003 \rightarrow \textit{forward single pointer}[\textit{forward}[0]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

0x8004 $\rightarrow$ *forward single pointer*[*forward*[0]] (1 $\rightarrow$ *byte*) = 00000000[8 bits]

$$0x8005 \rightarrow \textit{forward single pointer}[\textit{forward}[0]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x8006 \rightarrow \textit{forward single pointer}[\textit{forward}[0]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

$0x8007 \rightarrow \text{forward single pointer}[\text{forward}[0]] \ (1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$$0x8008 \rightarrow \textit{forward single pointer}[\textit{forward}[1]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x8009 \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800A \rightarrow \textit{forward single pointer}[\textit{forward}[1]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800B \rightarrow \textit{forward single pointer}[\textit{forward}[1]] (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800C \rightarrow \text{forward single pointer}[\text{forward}[1]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$$
$$0x800D \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$
$$0x800E \rightarrow \textit{forward single pointer}[\textit{forward}[1]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

0x800F $\rightarrow$ *forward single pointer*[*forward*[1]] (1 $\rightarrow$ *byte*) = 00000000[8 bits]

$0x8010 \rightarrow \textit{forward single pointer}[\textit{forward}[2]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

$$0x8011 \rightarrow \textit{forward single pointer}[\textit{forward}[2]](1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$$

$0x8012 \rightarrow \textit{forward single pointer}[\textit{forward}[2]] \ (1 \rightarrow \textit{byte}) = 00000000[8 \textit{ bits}]$

$0x8013 \rightarrow \textit{forward_single_pointer}[\textit{forward}[2]] \ (1 \rightarrow \textit{byte}) = 00000000[8\textit{ bits}]$

$0x8015 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8016 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8017 \rightarrow \text{forward single pointer}[\text{forward}[2]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8018 \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x8019 \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801A \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801B \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801C \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801D \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801E \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

$0x801F \rightarrow \text{forward single pointer}[\text{forward}[3]](1 \rightarrow \text{byte}) = 00000000[8 \text{ bits}]$

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
$0x8000 - 0x8007$	$0 - 7$	8 bytes	$\text{forward}[0]$	<i>Pointer variable store 8 byte address value.</i>
$0x8008 - 0x800F$	$8 - 15$	8 bytes	$\text{forward}[1]$	<i>Pointer variable store 8 byte address value.</i>
$0x8010 - 0x8017$	$16 - 23$	8 bytes	$\text{forward}[2]$	<i>Pointer variable store 8 byte address value.</i>
$0x8018 - 0x801F$	$24 - 31$	8 bytes	$\text{forward}[3]$	<i>Pointer variable store 8 byte address value.</i>

Now , why so?

*As it is **64 – bit architecture**. 8 bytes = (8 bytes $\times$ 8 bits) = 64 bit . $\left[\begin{array}{l} \text{As, 1 byte} \\ = 8 \text{ bits} \end{array} \right]$*

When 4 byte for integer is added which is (4 byte $\times$ 8 bits) = 32 bits .

*It will add another 8 byte padding – to make it (8 bytes $\times$ 8 bits) = 64 bit ,
if and only if the 4 byte integer is followed by 8 byte pointer to make
8 byte alignment.*

More accurately:

On a typical 64 – bit architecture, an 8 – byte pointer occupies 64 bits.

*A 4 – byte integer occupies 32 bits. If the integer is followed by an 8 – byte
–aligned pointer, the compiler may insert 4 bytes of padding so that
the pointer begins at an 8 – byte – aligned offset.*

For 32 – bit architecture ,

Pointer Variable → 4 bytes [32 bit]

Integer Variable → 4 bytes [32 bit]

Say, **key** is = 20

Decimal = (20)₁₀

Hexadecimal = (20)₁₀ = (0x14)₁₆

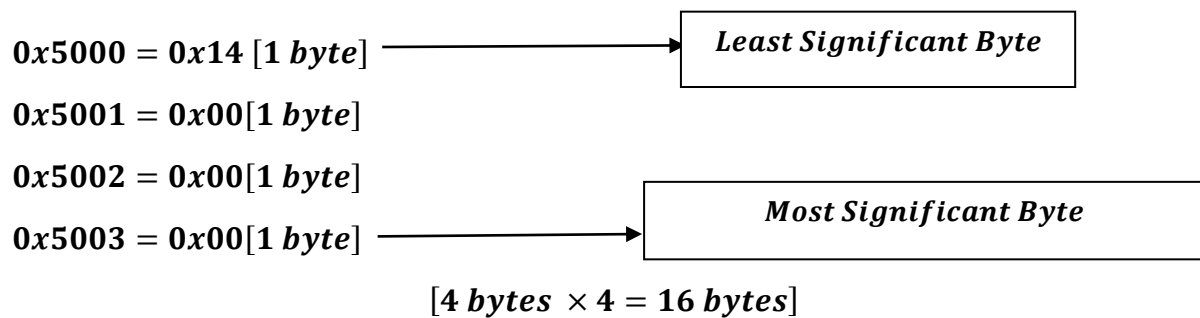
Say int **key** starts with address : (0x5000)₁₆

$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = (20480)_{10} \Rightarrow \frac{(20480)_{10}}{4 \text{ bytes}} \text{ i. e.}$$

$$\frac{(20480)_{10}}{(4)_{10}} = 5120_{10} \text{ and remainder : 0 , hence divisible by 4 bytes ,}$$

∴ Data value will get store in (0x5000)₁₆.

In 4 byte(32 bit) hexadecimal := 0x00 00 00 14



Represented in 8 bits as:

0x5000 = 00010100 [8 bits]

0x5001 = 00000000 [8 bits]

0x5002 = 00000000 [8 bits]

0x5003 = 00000000 [8 bits]

$$[8 \text{ bits} \times 4 = 32 \text{ bits}]$$

Next we have 4 – byte pointer variable , hence :

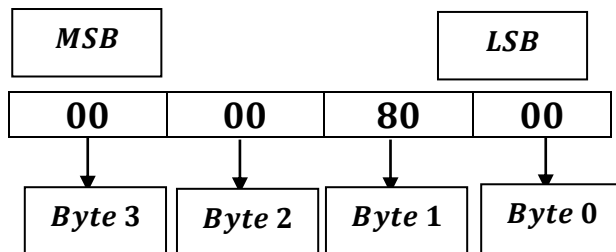
$$\therefore 5 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 4 \times 16^0 = (20484)_{10}$$

$$\Rightarrow \frac{(20484)_{10}}{4 \text{ bytes}} \text{ i.e. } \frac{(20484)_{10}}{(4)_{10}} = 5121_{10} \text{ and}$$

remainder : 0 , hence divisible by 4 bytes .

And it will store : 0x8000 [4 byte address]

0x8000 = 0x00 00 80 00 [4 hexadecimal digits]



0x5004 → forward double pointer (1 → byte) = 0x00

0x5005 → forward double pointer (1 → byte) = 0x80

0x5006 → forward double pointer (1 → byte) = 0x00

0x5007 → forward double pointer (1 → byte) = 0x00

*Least
Significant
Byte*

*Most
Significant
Byte*

Represented in 8 bit format as :

0x5004 → forward double pointer (1 → byte) = 00000000[8 bits]

0x5005 → forward double pointer (1 → byte) = 1000 0000 [8 bits]

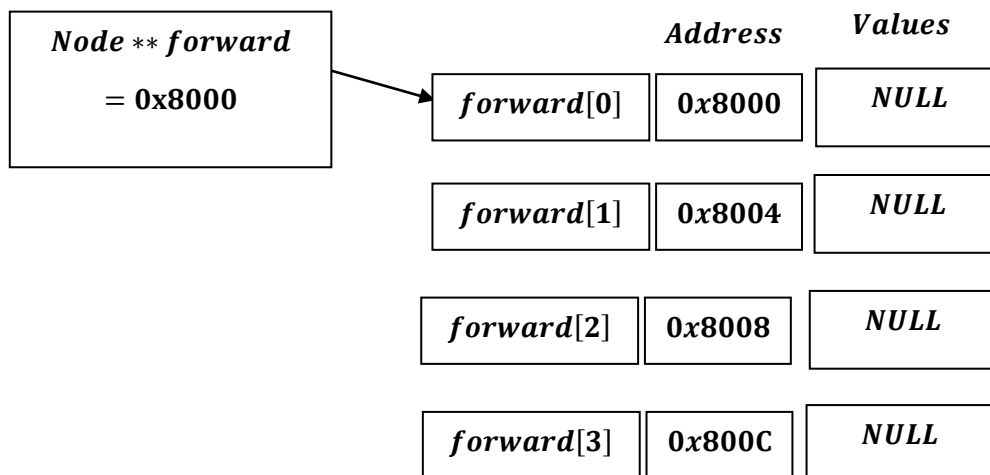
0x5006 → forward double pointer (1 → byte) = 00000000[8 bits]

0x5007 → forward double pointer (1 → byte) = 00000000[8 bits]

Now , we can say that:

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x5000 – 0x5003	0 – 3	4 bytes	<i>Integer : key</i>	<i>Integer value(Key)</i>
0x5004 – 0x5007	4 – 7	4 bytes	<i>double pointer (pointer to Pointer) variable named Forward</i>	<i>Address of forward[0] which is a single pointer variable.</i>

Now , when we come to:



0x8000 is divisible by 4 (byte) hence:

0x8000 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8001 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8002 → forward single pointer[forward[0]] (1 → byte) = 0x00

0x8003 → forward single pointer[forward[0]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8000 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8001 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8002 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8003 → forward single pointer[forward[0]] (1 → byte) = 00000000[8 bits]

0x8004 is divisible by 4 (byte) hence:

0x8004 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8005 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8006 → forward single pointer[forward[1]] (1 → byte) = 0x00

0x8007 → forward single pointer[forward[1]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8004 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8005 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8006 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

0x8007 → forward single pointer[forward[1]] (1 → byte) = 00000000[8 bits]

Again, 0x8008 is divisible by 4 (bytes) hence:

0x8008 → forward single pointer[forward[2]] (1 → byte) = 0x00

0x8009 → forward single pointer[forward[2]] (1 → byte) = 0x00

0x800A → forward single pointer[forward[2]] (1 → byte) = 0x00

0x800B → forward single pointer[forward[2]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x8008 → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x8009 → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x800A → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

0x800B → forward single pointer[forward[2]] (1 → byte) = 00000000[8 bits]

Again, 0x801C is divisible by 4 (bytes) hence:

0x800C → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800D → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800E → forward single pointer[forward[3]] (1 → byte) = 0x00

0x800F → forward single pointer[forward[3]] (1 → byte) = 0x00

Represented in 8 bit format as :

0x800C → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800D → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800E → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

0x800F → forward single pointer[forward[3]] (1 → byte) = 00000000[8 bits]

<i>Addresses</i>	<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0x8000 – 0x8003	0 – 3	4 bytes	<i>forward[0]</i>	<i>Pointer variable store 8 byte address value.</i>
0x8004 – 0x8007	4 – 7	4 bytes	<i>forward[1]</i>	<i>Pointer variable store 4 byte address value.</i>
0x8008 – 0x800B	8 – 11	4 bytes	<i>forward[2]</i>	<i>Pointer variable store 4 byte address value.</i>
0x800C – 0x800F	12 – 15	4 bytes	<i>forward[3]</i>	<i>Pointer variable store 4 byte address value.</i>

On a typical 32 – bit architecture, a 4 – byte pointer occupies 32 bits.

A 4 – byte integer also occupies 32 bits. If the integer is followed by a 4 – byte – aligned pointer, no padding is normally required because the 4 – byte integer already ends at an offset that is aligned to 4 bytes. Thus, the pointer can immediately follow the integer while maintaining 4 – byte alignment.
